Variables , fragmentos y bloques

Variables locales con th:with en Thymeleaf
- 1. Sintaxis básica
- 2. Crear una variable a partir de datos del modelo
- 3. Definir varias variables
- 4. Alcance de las variables
  - Una variable con el mismo nombre que un atributo del modelo
- 5. Usar th:with y th:text en el mismo elemento
- 6. Simplificar el acceso a un objeto
- 7. Combinar th:with con condiciones
  - Precaución con atributos en el mismo elemento
- 8. Combinar th:with con un bucle
- 9. Relación con los fragmentos
- 10. ¿Cuándo conviene utilizarlo?
- 11. Ejercicios prácticos

Uso de fragmentos en Thymeleaf
- 1. ¿Qué es un fragmento?
- 2. Organización de las plantillas
- 3. Definir los fragmentos
- 4. Incorporar fragmentos con th:replace
- 5. Reutilizar los fragmentos en otra página
- 6. Diferencia entre th:replace y th:insert
- 7. Fragmentos con parámetros
- 8. Pasar datos del modelo como parámetros
- 9. Errores habituales
- 10. Ejercicios prácticos

Uso de th:block en Thymeleaf
- 1. Sintaxis básica
- 2. Diferencia entre y <th:block>
- 3. Mostrar varios elementos con una condición
- 4. Repetir varios elementos con th:each
- 5. Generar varias filas de una tabla por cada objeto
- 6. Definir variables locales con th:with
  - Combinar variables y condiciones
- 7. Incorporar fragmentos
  - Condicionar la inclusión de un fragmento
- 8. Agrupar recursos comunes
- 9. Errores habituales
- 10. Ejercicios prácticos

Variables locales con `th:with` en Thymeleaf

Al elaborar una plantilla, a veces necesitamos guardar temporalmente un cálculo o dar un nombre sencillo a una expresión que vamos a utilizar varias veces.

El atributo `th:with` permite definir variables locales dentro de una plantilla Thymeleaf. Estas variables tienen un alcance limitado al elemento donde se declaran y a su contenido. thymeleaf.org

1. Sintaxis básica

```
<div th:with="nombre='Ana'">
   <p th:text="${nombre}">Nombre</p>
</div>
```

En este ejemplo:

- `th:with` define la variable.
- `nombre` es el nombre que le asignamos.
- `'Ana'` es su valor.
- `${nombre}` permite consultar ese valor.

El HTML generado será:

```
<div>
   <p>Ana</p>
</div>
```

No es necesario declarar un tipo como `String`, `int` o `double`.

Estas variables se procesan en el servidor al generar el HTML. No son variables JavaScript del navegador.

2. Crear una variable a partir de datos del modelo

Supongamos que la controladora envía el precio y la cantidad de un producto:

```
@GetMapping("/compra")
public String compra(Model model) {

   model.addAttribute("precio", 25);
   model.addAttribute("cantidad", 3);

   return "compra";
}
```

En `compra.html`, podemos calcular el importe:

```
<div th:with="importe=${precio * cantidad}">

    <p>
        Precio:
        <span th:text="${precio}"></span> €
    </p>

    <p>
        Cantidad:
        <span th:text="${cantidad}"></span>
    </p>

    <p>
        Importe:
        <span th:text="${importe}"></span> €
    </p>

</div>
```

La página mostrará:

```
Precio: 25 €
Cantidad: 3
Importe: 75 €
```

`precio` y `cantidad` proceden del modelo. `importe` se define localmente en la plantilla.

3. Definir varias variables

Podemos declarar varias variables en el mismo `th:with`, separándolas mediante comas. Una declaración posterior puede utilizar una variable definida antes en ese mismo atributo. thymeleaf.org

```
<div th:with="precio=20, cantidad=4, importe=${precio * cantidad}">

    <p th:text="|Precio: ${precio} €|"></p>
    <p th:text="|Cantidad: ${cantidad}|"></p>
    <p th:text="|Importe: ${importe} €|"></p>

</div>
```

Resultado:

```
Precio: 20 €
Cantidad: 4
Importe: 80 €
```

En este caso, todos los valores se han definido en la plantilla para simplificar el ejemplo.

En una aplicación, el precio y la cantidad normalmente procederían de los datos enviados por la controladora.

4. Alcance de las variables

Una variable local no está disponible en toda la página.

Observa este ejemplo:

```
<section th:with="mensaje='Bienvenido a la aplicación'">

   <h2 th:text="${mensaje}"></h2>

   <div>
      <p th:text="${mensaje}"></p>
   </div>

</section>

<p>Este párrafo está fuera del alcance de la variable.</p>
```

Podemos utilizar `mensaje` en el `<h2>` y en el `<p>` interior porque ambos están dentro de `<section>`.

Fuera de esa sección, la variable local `mensaje` ya no está disponible.

Conviene declarar la variable en el elemento más cercano que englobe todos los lugares donde se necesita.

Una variable con el mismo nombre que un atributo del modelo

Si la controladora envía:

```
model.addAttribute("nombre", "Luis");
```

Y la plantilla contiene:

```
<p th:text="${nombre}"></p>
<div th:with="nombre='Ana'">
   <p th:text="${nombre}"></p>
</div>
<p th:text="${nombre}"></p>
```

El resultado será:

```
Luis
Ana
Luis
```

La variable local oculta temporalmente el valor del modelo dentro del `<div>`. **No cambia el atributo original enviado por la controladora.**

Para facilitar la lectura, es preferible utilizar nombres diferentes cuando representen datos distintos.

5. Usar `th:with` y `th:text` en el mismo elemento

No siempre necesitamos crear un contenedor.

```
<p th:with="resultado=${8 * 5}"
   th:text="|El resultado es ${resultado}|">
</p>
```

Resultado:

El resultado es 40

Esto funciona porque Thymeleaf procesa `th:with` antes de `th:text`, independientemente del orden en que escribamos esos atributos. thymeleaf.org

En un cálculo tan corto también podríamos escribir directamente:

```
<p th:text="|El resultado es ${8 * 5}|"></p>
```

`th:with` resulta más útil cuando el nombre de la variable mejora la comprensión o cuando utilizamos el resultado varias veces.

6. Simplificar el acceso a un objeto

Imagina que la controladora envía un objeto `alumno` que contiene otro objeto llamado `direccion`, con las propiedades `calle`, `ciudad` y `codigoPostal`.

Podríamos escribir:

```
<p th:text="${alumno.direccion.calle}"></p>
<p th:text="${alumno.direccion.ciudad}"></p>
<p th:text="${alumno.direccion.codigoPostal}"></p>
```

Con una variable local:

```
<div th:with="direccion=${alumno.direccion}">
    <p th:text="${direccion.calle}"></p>
    <p th:text="${direccion.ciudad}"></p>
    <p th:text="${direccion.codigoPostal}"></p>
</div>
```

Así evitamos repetir la ruta `alumno.direccion`.

Este ejemplo supone que tanto `alumno` como su `direccion` existen.

7. Combinar `th:with` con condiciones

Podemos guardar el resultado de una comparación para utilizarlo en varios elementos.

Supongamos que el modelo contiene un atributo `nota`:

```
model.addAttribute("nota", 7);
```

En la plantilla:

```
<div th:with="aprobado=${nota >= 5}">

   <p th:text="|Nota obtenida: ${nota}|"></p>

   <p th:if="${aprobado}">
      Has aprobado.
   </p>

   <p th:unless="${aprobado}">
      Debes recuperar la actividad.
   </p>

</div>
```

Precaución con atributos en el mismo elemento

No debemos utilizar una variable recién creada por `th:with` en un `th:if` del mismo elemento:

```
<!-- Incorrecto: th:if se procesa antes que th:with -->
<p th:with="aprobado=${nota >= 5}"  th:if="${aprobado}">
   Has aprobado.
</p>
```

Thymeleaf evalúa `th:if` antes de crear las variables de `th:with`. Cambiar el orden escrito de los atributos no lo soluciona. thymeleaf.org

La solución es colocar la declaración en un elemento exterior:

```
<div th:with="aprobado=${nota >= 5}">
   <p th:if="${aprobado}">Has aprobado.</p>
</div>
```

8. Combinar `th:with` con un bucle

Supongamos que el modelo contiene una lista `lineas`, cuyos objetos tienen las propiedades `nombre`, `precio` y `cantidad`.

```
<table>
   <thead>
      <tr>
         <th>Producto</th>
         <th>Precio</th>
         <th>Cantidad</th>
         <th>Importe</th>
      </tr>
   </thead>

   <tbody>
      <tr th:each="linea : ${lineas}"
          th:with="importe=${linea.precio * linea.cantidad}">
         <td th:text="${linea.nombre}"></td>
         <td th:text="${linea.precio}"></td>
         <td th:text="${linea.cantidad}"></td>
         <td th:text="${importe}"></td>
      </tr>
   </tbody>
</table>
```

Cada fila utiliza su propio `importe`. Aquí podemos combinar ambos atributos porque `th:each` se procesa antes que `th:with`. thymeleaf.org

No debemos utilizar este mecanismo como si fuera un acumulador Java para sumar todas las filas. Si necesitamos el total de la compra, podemos calcularlo en Java y enviarlo al modelo.

9. Relación con los fragmentos

Podemos calcular un valor local y pasarlo a un fragmento situado dentro de ese ámbito.

Por ejemplo, en `fragmentos/comunes.html`:

```
<p th:fragment="aviso(texto)" th:text="${texto}">
   Mensaje de ejemplo
</p>
```

En la página principal:

```
<div th:with="mensaje=|Bienvenido, ${nombreUsuario}|">
   <div th:replace="~{fragmentos/comunes :: aviso(${mensaje})}">
   </div>
</div>
```

Colocamos `th:with` en el contenedor exterior porque la inclusión de fragmentos se procesa antes que la definición de variables locales cuando ambas están en la misma etiqueta. thymeleaf.org

10. ¿Cuándo conviene utilizarlo?

Situación	Uso propuesto
Una expresión se repite varias veces.	Guardar su resultado con un nombre descriptivo.
Una ruta de acceso a propiedades es larga.	Crear una referencia local más corta.
Un cálculo sencillo ayuda a presentar datos.	Definir una variable para mostrarlo.
Una operación contiene reglas de negocio complejas.	Resolverla en Java y enviar el resultado al modelo.
Un valor debe conservarse entre peticiones.	Gestionarlo fuera de `th:with`, según las necesidades de la aplicación.

Los cálculos de estos ejemplos sirven para practicar la sintaxis. En una aplicación real, decisiones como los descuentos aplicables o el importe definitivo de una factura deben resolverse en la lógica de la aplicación.

11. Ejercicios prácticos

Ejercicio 1. Mensaje de bienvenida

Define con `th:with` las variables `nombre` y `curso`. Utilízalas para mostrar:

Hola, Marta. Estás matriculada en 2.º de DAW.

Ejercicio 2. Importe de una compra

Desde una controladora, envía al modelo:

- `precio`: 15.
- `cantidad`: 4.

Calcula `importe` mediante `th:with` y muestra los tres valores.

Ejercicio 3. Comprobar el alcance

Partiendo de este código:

```
<p th:text="${ciudad}"></p>

<div th:with="ciudad='Madrid'">
   <p th:text="${ciudad}"></p>
</div>

<p th:text="${ciudad}"></p>
```

La controladora ha enviado `ciudad` con el valor `"Toledo"`.

Indica qué mostrará cada párrafo y explica por qué.

Ejercicio 4. Resultado académico

Envía una nota desde la controladora. Crea una variable local `aprobado` y muestra:

- «Actividad superada» si la nota es mayor o igual que 5.
- «Actividad pendiente» en caso contrario.

Ejercicio 5. Localizar y corregir un error

Explica por qué este código no puede utilizar correctamente la variable local `esMayor` en la condición y corrígelo:

```
<p th:with="esMayor=${edad >= 18}"
    th:if="${esMayor}">
   Acceso permitido.
</p>
```

Ejercicio 6. Fragmento con mensaje calculado

Crea un fragmento `resultado(texto)`. En la vista principal, utiliza `th:with` para construir un mensaje a partir de un nombre y una nota recibidos del modelo. Pasa ese mensaje al fragmento.

Uso de fragmentos en Thymeleaf

En una aplicación web, muchas páginas comparten elementos: la cabecera, el menú de navegación, el pie de página o los enlaces a las hojas de estilo.

Si copiamos estos elementos en cada plantilla, cualquier modificación nos obliga a editar varios archivos. **Los fragmentos permiten definir una parte de la página una sola vez y reutilizarla en distintas plantillas.**

Al terminar este contenido, podrás crear fragmentos, incorporarlos a otras páginas y pasarles parámetros para adaptar su contenido.

1. ¿Qué es un fragmento?

Un **fragmento** es una parte de una plantilla HTML que podemos identificar y reutilizar.

Por ejemplo, podemos dividir las páginas de una aplicación en:

- Una cabecera común.
- Un menú de navegación común.
- El contenido específico de cada página.
- Un pie de página común.

Para trabajar con fragmentos utilizaremos estos atributos:

Atributo	Función
`th:fragment`	Define un fragmento y le asigna un nombre.
`th:replace`	Sustituye el elemento que lo contiene por el fragmento.
`th:insert`	Introduce el fragmento dentro del elemento que lo contiene.

Estos mecanismos forman parte de Thymeleaf y permiten reutilizar bloques de una plantilla en otras. Thymeleaf

2. Organización de las plantillas

Vamos a trabajar con una aplicación que tenga dos páginas: **Inicio** y **Productos**.

Dentro de `src/main/resources/templates`, crearemos estos archivos:

Archivo	Contenido
fragmentos/comunes.html	Cabecera, menú y pie de página.
inicio.html	Página de inicio.
productos.html	Página de productos.

La carpeta `fragmentos` nos ayuda a organizar el proyecto. Su nombre es una elección nuestra, no una palabra reservada.

3. Definir los fragmentos

Creamos el archivo `fragmentos/comunes.html`:

```
<!DOCTYPE html>
<html lang="es" xmlns:th="http://www.thymeleaf.org">
<head>
    <meta charset="UTF-8">
    <title>Elementos comunes</title>
</head>
<body>

    <header th:fragment="cabecera">
        <h1>Tienda DAW</h1>
        <p>Tu tienda de informática y pescadería</p>
    </header>

    <nav th:fragment="menu">
        <a th:href="@{/}">Inicio</a>
        <a th:href="@{/productos}">Productos</a>
    </nav>

    <footer th:fragment="pie">
        <p>Aplicación de ejemplo de 2.º de DAW</p>
    </footer>

</body>
</html>
```

Hemos definido **tres fragmentos en un mismo archivo**:

- `cabecera`: incluye el elemento `<header>` y su contenido.
- `menu`: incluye el elemento `<nav>` y sus enlaces.
- `pie`: incluye el elemento `<footer>` y su contenido.

El nombre del archivo y los nombres de los fragmentos son distintos: el archivo es `comunes.html`, mientras que uno de sus fragmentos se llama `cabecera`.

4. Incorporar fragmentos con `th:replace`

Creamos `inicio.html`:

```
<!DOCTYPE html>
<html lang="es" xmlns:th="http://www.thymeleaf.org">
<head>
   <meta charset="UTF-8">
   <title>Inicio</title>
</head>
<body>

   <div th:replace="~{fragmentos/comunes :: cabecera}"></div>

   <div th:replace="~{fragmentos/comunes :: menu}"></div>

   <main>
      <h2>Bienvenido</h2>
      <p>Consulta nuestros productos informáticos.</p>
   </main>

   <div th:replace="~{fragmentos/comunes :: pie}"></div>

</body>
</html>
```

La expresión:

```
~{fragmentos/comunes :: cabecera}
```

se interpreta así:

Parte	Significado
`~{...}`	Expresión de fragmento.
`fragmentos/comunes`	Plantilla que contiene el fragmento.
`::`	Separa la plantilla del fragmento seleccionado.
`cabecera`	Nombre del fragmento.

Con la configuración habitual de Spring Boot, la ruta se expresa desde `templates`, sin escribir esa carpeta ni la extensión `.html`.

¿Qué HTML se obtiene?

Esta instrucción:

```
<div th:replace="~{fragmentos/comunes :: cabecera}"></div>
```

se convierte en:

```
<header>
  <h1>Tienda DAW</h1>
  <p>Tu tienda de informática</p>
</header>
```

El `<div>` original desaparece porque ha sido sustituido por el fragmento.

5. Reutilizar los fragmentos en otra página

Creamos `productos.html`:

```
<!DOCTYPE html>
<html lang="es" xmlns:th="http://www.thymeleaf.org">
<head>
  <meta charset="UTF-8">
  <title>Productos</title>
</head>
<body>

  <div th:replace="~{fragmentos/comunes :: cabecera}"></div>

  <div th:replace="~{fragmentos/comunes :: menu}"></div>

  <main>
    <h2>Nuestros productos</h2>

    <ul>
      <li>Teclado</li>
      <li>Ratón</li>
      <li>Monitor</li>
    </ul>
  </main>

  <div th:replace="~{fragmentos/comunes :: pie}"></div>

</body>
</html>
```

Ambas páginas comparten los mismos elementos. Si modificamos el pie en `comunes.html`, el cambio se verá en las dos páginas cuando vuelvan a renderizarse.

Para servir las vistas, podemos utilizar esta controladora, situada en el paquete correspondiente de nuestro proyecto:

```
import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.GetMapping;
```

```
@Controller
public class TiendaController {

    @GetMapping("/")
    public String inicio() {
        return "inicio";
    }

    @GetMapping("/productos")
    public String productos() {
        return "productos";
    }
}
```

El fragmento no necesita una ruta ni un método propio en la controladora. Thymeleaf lo incorpora cuando procesa la vista principal en el servidor.

Para probarlo, arrancamos la aplicación y accedemos a `http://localhost:8080/`, suponiendo que utilice el puerto 8080.

6. Diferencia entre `th:replace` y `th:insert`

Los dos atributos permiten incorporar un fragmento, pero generan una estructura HTML distinta. Thymeleaf

Con `th:replace`:

```
<div th:replace="~{fragmentos/comunes :: pie}"></div>
```

Resultado:

```
<footer>
    <p>Aplicación de ejemplo de 2.º de DAW</p>
</footer>
```

Con `th:insert`:

```
<div th:insert="~{fragmentos/comunes :: pie}"></div>
```

Resultado:

```
<div>
    <footer>
        <p>Aplicación de ejemplo de 2.º de DAW</p>
    </footer>
</div>
```

Utiliza `th:insert` cuando quieras conservar el contenedor, por ejemplo, porque tiene una clase CSS:

```
<div class="zona-inferior"
    th:insert="~{fragmentos/comunes :: pie}">
</div>
```

Con `th:replace`, ese `<div>` y su clase desaparecerían.

7. Fragmentos con parámetros

A veces queremos reutilizar la misma estructura, pero mostrar información diferente en cada página.

Podemos definir **parámetros** en el fragmento y proporcionar sus valores al incorporarlo. thymeleaf.org

Sustituimos la cabecera anterior de `comunes.html` por esta versión:

```
<header th:fragment="cabecera(titulo, subtitulo)">
   <h1 th:text="${titulo}">Títulodeejemplo</h1>
   <p th:text="${subtitulo}">Subtítulo de ejemplo</p>
</header>
```

Ahora la cabecera recibe dos valores:

- `titulo`.
- `subtitulo`.

En `inicio.html`, actualizamos la llamada:

```
<div th:replace="~{fragmentos/comunes :: cabecera(
   'Tienda DAW',
   'Bienvenido a nuestra tienda'
)}"></div>
```

En `productos.html`:

```
<div th:replace="~{fragmentos/comunes :: cabecera(
   'Nuestros productos',
   'Consulta el catálogo disponible'
)}"></div>
```

Debemos actualizar las dos páginas, porque hemos cambiado la definición del fragmento para que reciba parámetros.

8. Pasar datos del modelo como parámetros

Los valores también pueden proceder de la controladora.

Modificamos el método `productos()` y añadimos el import de `Model`:

```
import org.springframework.ui.Model;
@GetMapping("/productos")
public String productos(Model model) {

    model.addAttribute("tituloPagina", "Catálogo de informática");
    model.addAttribute("descripcion", "Productos para tu equipo");

    return "productos";
}
```

En `productos.html`:

```
<div th:replace="~{fragmentos/comunes :: cabecera(
            ${tituloPagina},
            ${descripcion}
           )}">
</div>
```

La relación entre los datos es:

Atributo del modelo	Parámetro del fragmento
`tituloPagina`	`titulo`
`descripcion`	`subtitulo`

Los nombres no tienen que coincidir: en esta llamada, los valores se asignan **por su posición**.

9. Errores habituales

Error	Cómo evitarlo
Confundir el archivo con el fragmento.	En `fragmentos/comunes :: menu`, `comunes` es el archivo y `menu` es el fragmento.
Escribir mal la ruta.	Comprueba la ubicación real del archivo dentro de `templates`.
Utilizar un nombre de fragmento inexistente.	Revisa el valor declarado en `th:fragment`.

Olvidar parámetros después de modificar un fragmento.	Actualiza todas las llamadas que lo utilizan.
Esperar que se conserve una clase del elemento sustituido.	Usa `th:insert` si necesitas conservar ese contenedor.
Abrir el HTML directamente y esperar que se procesen los fragmentos.	Accede a la página a través de la aplicación Spring Boot.

10. Ejercicios prácticos

Ejercicio 1. Un pie común

Crea un fragmento llamado `pie` que muestre el nombre de la aplicación y el grupo. Incorpóralo a dos páginas diferentes.

Modifica el texto del fragmento y comprueba que el cambio aparece en ambas páginas.

Ejercicio 2. Comparar inserción y sustitución

Incluye el mismo fragmento una vez con `th:insert` y otra con `th:replace`.

Utiliza el inspector del navegador para comprobar el HTML generado. Explica qué diferencia existe entre las dos estructuras.

Ejercicio 3. Cabecera personalizada

Crea un fragmento `cabecera(titulo)` y utilízalo en dos páginas:

- Una debe mostrar «Página de inicio».
- Otra debe mostrar «Listado de alumnos».

Ejercicio 4. Mensaje recibido del modelo

Crea un fragmento `aviso(mensaje)` que muestre un párrafo.

Desde la controladora, añade al modelo un atributo llamado `informacion` con el texto «La matrícula termina el viernes». Pasa ese atributo como parámetro al fragmento.

Ejercicio 5. Aplicación con tres vistas

Crea las páginas Inicio, Cursos y Contacto. Las tres deben compartir:

- Un menú de navegación.
- Una cabecera con un título diferente en cada página.
- Un pie de página.

Cada vista debe tener su propia ruta y contenido principal.

Uso de `th:block` en Thymeleaf

En una plantilla, a veces necesitamos aplicar una condición, repetir varios elementos o definir variables locales sin añadir un contenedor al HTML final.

`th:block` es un elemento especial de Thymeleaf que permite agrupar contenido. Al procesar la plantilla, Thymeleaf ejecuta sus atributos y elimina la etiqueta `th:block`, conservando el contenido que corresponda.

1. Sintaxis básica

A diferencia de `th:if`, `th:each` o `th:with`, **`th:block` es una etiqueta, no un atributo**.

```
<th:block>
   <h2>Información del curso</h2>
   <p>Desarrollo de Aplicaciones Web</p>
</th:block>
```

El HTML generado será:

```
<h2>Información del curso</h2>
<p>Desarrollo de Aplicaciones Web</p>
```

Las etiquetas `<th:block>` y `</th:block>` desaparecen.

Por sí solo, este bloque no aporta ningún comportamiento especial. Resulta útil cuando le añadimos atributos de Thymeleaf.

2. Diferencia entre `<div>` y `<th:block>`

Un `<div>` es un elemento HTML que permanece en la página. Un `<th:block>` sirve como contenedor durante el procesamiento de la plantilla, pero desaparece del resultado.

Utilizando `<div>`:

```
<div th:with="nombre='Ana'">
   <p th:text="${nombre}"></p>
</div>
```

Resultado:

```
<div>
   <p>Ana</p>
</div>
```

Utilizando `<th:block>`:

```
<th:block th:with="nombre='Ana'">
   <p th:text="${nombre}"></p>
</th:block>
```

Resultado:

```
<p>Ana</p>
```

Necesidad	Elemento apropiado
Crear un contenedor para aplicar estilos o distribuir elementos.	Un elemento HTML, como `<div>` o `<section>`.
Agrupar elementos únicamente para aplicar instrucciones de Thymeleaf.	`<th:block>`.

3. Mostrar varios elementos con una condición

Supongamos que la controladora envía estos datos:

```
@GetMapping("/alumno")
public String alumno(Model model) {

   model.addAttribute("nombre", "Lucía");
   model.addAttribute("matriculado", true);

   return "alumno";
}
```

Queremos mostrar un título, un mensaje y un enlace solamente si el alumno está matriculado:

```
<th:block th:if="${matriculado}">
   <h2>Matrícula confirmada</h2>
   <p th:text="|Bienvenida, ${nombre}|"></p>
   <a th:href="@{/cursos}">Consultar cursos</a>
</th:block>
```

Si matriculado vale true, el resultado será:

```
<h2>Matrícula confirmada</h2>
<p>Bienvenida, Lucía</p>
<a href="/cursos">Consultar cursos</a>
```

Si vale `false`, no se mostrará ninguno de esos elementos.

Una única condición controla todo el grupo, sin introducir un contenedor adicional.

También podemos definir el caso contrario:

```
<th:block th:unless="${matriculado}">
   <h2>Matrícula pendiente</h2>
   <p>Debes completar el proceso de matriculación.</p>
</th:block>
```

4. Repetir varios elementos con `th:each`

Cuando cada repetición corresponde a un solo elemento, podemos colocar `th:each` directamente en él:

```
<ul>
   <li th:each="nombre : ${nombres}"
      th:text="${nombre}">
   </li>
</ul>
```

Pero, ¿qué ocurre si queremos generar **varios elementos hermanos por cada dato**?

Supongamos que la controladora envía:

```
model.addAttribute(
   "asignaturas",
   List.of("Cliente", "Servidor", "Despliegue")
);
```

Para utilizar `List.of`, necesitamos:

```
import java.util.List;
```

En la plantilla:

```
<th:block th:each="asignatura : ${asignaturas}">

   <h2 th:text="${asignatura}"></h2>
   <p>Consulta los materiales y las actividades de esta asignatura.</p>
   <hr>

</th:block>
```

Por cada asignatura se generan un título, un párrafo y una línea horizontal:

```
<h2>Cliente</h2>
<p>Consulta los materiales y las actividades de esta asignatura.</p>
<hr>

<h2>Servidor</h2>
<p>Consulta los materiales y las actividades de esta asignatura.</p>
<hr>

<h2>Despliegue</h2>
```

```
<p>Consulta los materiales y las actividades de esta asignatura.</p>
<hr>
```

5. Generar varias filas de una tabla por cada objeto

Un uso práctico consiste en generar **dos filas por cada alumno**:

- Una fila con su nombre y curso.
- Otra fila con sus observaciones.

Suponemos que el modelo contiene una lista `alumnos`, cuyos objetos tienen las propiedades `nombre`, `curso` y `observaciones`.

```
<table>
   <thead>
      <tr>
         <th>Nombre</th>
         <th>Curso</th>
      </tr>
   </thead>

   <tbody>
      <th:block th:each="alumno : ${alumnos}">

         <tr>
            <td th:text="${alumno.nombre}"></td>
            <td th:text="${alumno.curso}"></td>
         </tr>

         <tr>
            <td colspan="2"
               th:text="|Observaciones: ${alumno.observaciones}|">
            </td>
         </tr>

      </th:block>
   </tbody>
</table>
```

Para un alumno, el resultado podría ser:

```
<tbody>
   <tr>
      <td>Lucía</td>
      <td>2.º DAW</td>
   </tr>
   <tr>
      <td colspan="2">
         Observaciones: Prácticas entregadas
      </td>
   </tr>
</tbody>
```

No utilizamos un `<div>` para agrupar las filas porque no es un contenedor válido para ellas dentro de `<tbody>`. Al desaparecer `th:block`, el HTML generado mantiene las filas directamente dentro del cuerpo de la tabla.

6. Definir variables locales con `th:with`

Podemos utilizar `th:block` para establecer el alcance de una variable sin añadir un elemento HTML.

```
<th:block th:with="precio=15, cantidad=4, importe=${precio * cantidad}">

    <p th:text="|Precio: ${precio} €|"></p>
    <p th:text="|Cantidad: ${cantidad}|"></p>
    <p th:text="|Importe: ${importe} €|"></p>

</th:block>
```

Resultado:

```
<p>Precio: 15 €</p>
<p>Cantidad: 4</p>
<p>Importe: 60 €</p>
```

Las variables locales siguen teniendo un alcance limitado al bloque, aunque este desaparezca del HTML final.

Combinar variables y condiciones

Como vimos en el contenido de `th:with`, conviene separar la declaración de una variable y la condición que la utiliza:

```
<th:block th:with="aprobado=${nota >= 5}">

    <p th:if="${aprobado}">Actividad superada.</p>
    <p th:unless="${aprobado}">Actividad pendiente.</p>

</th:block>
```

El bloque exterior define la variable y los elementos interiores la consultan.

7. Incorporar fragmentos

También podemos utilizar `th:block` como punto de inserción de un fragmento.

Supongamos que `fragmentos/comunes.html` contiene:

```
<footer th:fragment="pie">
    <p>Aplicación de 2.º de DAW</p>
</footer>
```

En otra plantilla:

```
<th:block th:replace="~{fragmentos/comunes :: pie}"></th:block>
```

Resultado:

```
<footer>
   <p>Aplicación de 2.º de DAW</p>
</footer>
```

Con `th:replace`, también podríamos utilizar un `<div>`, porque sería sustituido igualmente:

```
<div th:replace="~{fragmentos/comunes :: pie}"></div>
```

En este caso, las dos opciones producen el mismo resultado.

Condicionar la inclusión de un fragmento

Para mostrar el pie solo cuando una condición sea verdadera:

```
<th:block th:if="${mostrarPie}">
   <th:block th:replace="~{fragmentos/comunes :: pie}"></th:block>
</th:block>
```

El bloque exterior evalúa la condición y el interior incorpora el fragmento.

Esta estructura evita combinar en la misma etiqueta la condición y la sustitución, cuyo orden de procesamiento podría dar un resultado distinto al esperado.

8. Agrupar recursos comunes

Podemos crear un fragmento que reúna varias etiquetas, como enlaces a hojas de estilo.

En `fragmentos/recursos.html`:

```
<!DOCTYPE html>
<html lang="es" xmlns:th="http://www.thymeleaf.org">
<head>

   <th:block th:fragment="estilos">
      <link rel="stylesheet" th:href="@{/css/general.css}">
      <link rel="stylesheet" th:href="@{/css/tablas.css}">
   </th:block>

   <title>Recursos comunes</title>
</head>
<body>
</body>
</html>
```

En la página que utiliza estos recursos:

```
<head>
   <meta charset="UTF-8">
   <title>Listado de alumnos</title>

   <th:block th:replace="~{fragmentos/recursos :: estilos}"></th:block>
</head>
```

El resultado incluirá las dos etiquetas `<link>`, sin un contenedor alrededor.

Los archivos CSS del ejemplo se guardarían en:

- `src/main/resources/static/css/general.css`
- `src/main/resources/static/css/tablas.css`

9. Errores habituales

Error	Explicación o solución
Escribir `<div th:block>`.	`th:block` se utiliza como etiqueta: `<th:block>...</th:block>`.
Añadir una clase CSS a `th:block` esperando un contenedor visible.	El bloque desaparece. Aplica la clase a un elemento HTML que permanezca.
Utilizarlo para agrupar visualmente una sección.	Usa `<div>`, `<section>` u otro elemento HTML apropiado.
Consultar fuera del bloque una variable declarada dentro.	La variable local ya no está disponible fuera de su alcance.
Abrir la plantilla directamente y esperar su procesamiento.	Accede a ella a través de la aplicación Spring Boot.

Por ejemplo, este código no genera un contenedor con borde:

```
<th:block class="con-borde">
   <p>Información del alumno</p>
</th:block>
```

Para ello debemos utilizar un elemento real:

```
<div class="con-borde">
   <p>Información del alumno</p>
</div>
```

10. Ejercicios prácticos

Ejercicio 1. Aviso de matrícula

Envía desde la controladora un atributo booleano llamado `plazoAbierto`.

Utiliza un único `th:block` para mostrar, cuando sea verdadero:

- El título «Matrícula abierta».
- Un párrafo informativo.
- Un enlace a `/matricula`.

Ejercicio 2. Listado de asignaturas

Envía al modelo una lista de nombres de asignaturas.

Por cada asignatura, genera un <h2> con su nombre y un <p> con el texto «Materiales disponibles próximamente». Utiliza `th:block` para repetir ambos elementos.

Ejercicio 3. Variables locales

Recibe del modelo los atributos `base` y `altura`.

Dentro de un `th:block`, define una variable `area` que calcule el área de un rectángulo. Muestra la base, la altura y el área en tres párrafos.

Ejercicio 4. Interpretar el resultado

Indica el HTML que se generará:

```
<th:block th:with="nombre='Carlos', nota=8">
   <h2 th:text="${nombre}"></h2>
   <p th:text="|Nota: ${nota}|"></p>
   <p th:if="${nota >= 5}">Aprobado</p>
</th:block>
```

Ejercicio 5. Fragmento de recursos

Crea un fragmento llamado `scripts` mediante `th:block`. Debe contener dos etiquetas `<script>` que carguen archivos JavaScript de tu aplicación.

Inclúyelo al final del `<body>` de dos páginas distintas y comprueba que en el HTML generado aparecen los scripts, pero no la etiqueta `th:block`.